Fine-Grained DINO Tuning with Dual Supervision for Face Forgery Detection: A Reproduction Study

Rohan Sen-M26DS010, Saksham Kohli-M26DS011, Gobinda Prasad Bag-M26DS016

***Abstract*—Deepfakes are making it harder to trust what we see, and a useful detector has to keep working on forgeries it never saw during training. Zhang et al. [1] tackle this with the DeepFake Fine-Grained Adapter (DFF-Adapter). Their idea is to keep DINOv2 frozen, place small routed low-rank (LoRA) experts in every Transformer block, and train the model on two tasks at once: deciding whether a face is real or fake, and naming the method used to fake it. A shared adapter branch lets the two tasks help each other. Because the authors have not released their code, we wrote the whole method ourselves in PyTorch, prepared FaceForensics++ (c23) with the DeepfakeBench pipeline, and trained on the official FF++ splits. Using the DINOv2-with-registers-large backbone, our model has 3.55M trainable parameters, which matches the 3.5M given in the paper. On the held-out FF++ test split it reaches a video-level AUC of 99.62%, slightly above the 99.56% reported by the authors. On four unseen datasets (Celeb-DF v1 and v2, DFDCP and DFDC) it averages 88.00%, which is 5.24 points below the paper's 93.23%, and on twelve manipulation methods from DF40 it averages 94.16% against the paper's 95.20%. In short, the in-domain result reproduces fully, the cross-manipulation result comes close, and part of the cross-dataset generalisation is still missing. We list every place where our version differs from the paper, look inside the trained adapters, and use the authors' own ablations to suggest where the remaining gap may come from.**

***Index Terms*—Deepfake detection, face forgery, DINOv2, LoRA, mixture of experts, parameter-efficient fine-tuning, multi-task learning, cross-dataset generalisation, reproducibility.**

I. Introduction

FACE manipulations made by modern generative models are now good enough to fool most people. This opens the door to identity theft, privacy abuse and the spread of misinformation [2], [3]. A recurring problem is that a detector trained on one dataset often breaks down when it meets a forgery method it has not seen before, which makes generalisation the main open question in this area.

Many recent detectors start from a large pre-trained vision model and fine-tune it. DINOv2 [4] is a natural choice, since its self-supervised features keep fine details of local texture and geometry. Most DINOv2-based detectors, though, treat the task as a plain real-versus-fake problem and only adapt the last few layers. In doing so they overlook something important: each forgery method leaves its own kind of trace.

The DFF-Adapter of Zhang et al. [1] is built around this observation. It (i) adds routed multi-head LoRA experts to every Transformer block of a frozen DINOv2, (ii) trains a second classifier that predicts the forgery type next to the main authenticity detector, and (iii) uses a shared adapter branch to pass these fine-grained, method-specific cues on to the authenticity head (Fig. 1).

The authors are with the Indian Institute of Technology Bhilai, India (e-mail: m26ds010@iitbhilai.ac.in; m26ds011@iitbhilai.ac.in; m26ds016@iitbhilai.ac.in).

A. Project Goal

Our project follows the criterion of reproducing a paper's results when no code is available. As set out in our statement of purpose, we aimed to (1) write the method from scratch; (2) put together a data and training pipeline on FaceForensics++ (c23) that others can rerun; and (3) match the paper's intra-dataset AUC of roughly 99% and, in a later phase, its average cross-dataset AUC of roughly 93%.

B. Contributions

- Our own PyTorch implementation of the DFF-Adapter, built on Hugging Face `Dinov2WithRegistersModel`. It covers the forgery-aware multi-head router, the shared head adapter and the two-pass training loop.
- A full data pipeline, from the official FF++ c23 download, through DeepfakeBench face alignment and cropping, to a frame-level loader that follows the official train/val/test video pairs.
- Two training runs (ViT-B for 12 epochs and ViT-L for 50 epochs), a parameter count that we checked line by line against the paper, and a look at what the trained adapters actually learned.
- A full evaluation of the ViT-L model on the FF++ test split, on four cross-dataset benchmarks and on twelve DF40 manipulation methods, compared method by method with the paper.

C. Team Contributions

- **Rohan Sen** (M26DS010, m26ds010@iitbhilai.ac.in): data preprocessing, and setting up the FaceForensics++ and cross-dataset evaluation pipelines.
- **Saksham Kohli** (M26DS011, m26ds011@iitbhilai.ac.in): baseline DINOv2 integration and the DFF-Adapter architecture, including the forgery-aware multi-head router and the shared-enhanced task fusion.
- **Gobinda Prasad Bag** (M26DS016, m26ds016@iitbhilai.ac.in): the dual-branch training loop, the loss functions, hyperparameter tuning and the evaluation metrics.

II. Related Work and Literature Review

A. Classical Forgery Detection

The first detectors searched for visible flaws, such as warped regions or the boundary where a swapped face is blended in [5]. As generators got better, researchers turned to less obvious signals in the frequency domain and in noise residuals [6], and to synthetic training data that mimics blending artefacts, for example self-blended images [7]. Even so, these methods tend to overfit to the manipulations they were trained on.

B. Pre-trained Large Models for Detection

Foundation models like DINOv2 come with broad visual knowledge that can be adapted cheaply through parameter-efficient fine-tuning. These models are Vision Transformers [8], built from the self-attention blocks of [9]; the variant we use adds register tokens, which give cleaner attention maps [10]. LoRA [11] adapts such a model by learning small low-rank updates on top of frozen weights. Sparse mixture-of-experts layers [12] let a router send each input to a few specialised sub-networks, and MoE-FFD [13] brings this idea to face forgery detection by mixing LoRA and adapters with expert routing.

C. Benchmarks

FaceForensics++ [14] is the standard training set. Cross-dataset generalisation is usually measured on Celeb-DF [15] and DFDC [16], and DF40 [17] adds forty newer manipulation methods. DeepfakeBench [18] gives all of these a common preprocessing and evaluation protocol. What sets the DFF-Adapter apart from earlier DINOv2 detectors is that it adapts every block and adds supervision on the type of manipulation.

III. Proposed Methodology

A. Overview

The DINOv2 weights are never changed. Instead, we wrap some of the linear layers inside the self-attention of each Transformer block with a DFF-Adapter module, which adds small trainable low-rank updates. Every module contains three expert paths: an *authenticity* path, a *forgery-type* path and a *shared* path (Fig. 2). A single global task flag tells the model which of the two task paths to use in a given forward pass. On top, the final CLS token is fed to a linear authenticity head (1 logit) and a linear forgery-type head (5 logits).

B. Forgery-Aware Multi-Head Router

Take $\mathbf{X} \in \mathbb{R}^{L\times d}$ to be the input of a wrapped linear layer, with L tokens of width d. We cut the channel axis into h slices $\mathbf{X}^{(k)} \in \mathbb{R}^{L\times d_h}$, where $d_h = d/h$. Each path has its own bank of N LoRA experts $\{(\mathbf{A}_j, \mathbf{B}_j)\}_{j=1}^{N}$, with $\mathbf{A}_j \in \mathbb{R}^{d_h\times r_h}$, $\mathbf{B}_j \in \mathbb{R}^{r_h\times d_h}$ and a per-slice rank of $r_h = r/h$. All h slices of a path draw on the same bank.

Task-specific head adapter: For task $t \in \{0, 1\}$ and slice k, a router gives each expert a score for every token. In our version these scores come from a per-slice projection of the token, $\mathbf{W}^r_{t,k} \in \mathbb{R}^{d_h\times N}$, plus a learned bias $\mathbf{z}_{t,k} \in \mathbb{R}^N$:

$$g_{t,k} = \sigma\Big(\mathbf{X}^{(k)}\mathbf{W}^r_{t,k} + \mathbf{z}_{t,k}\Big), \tag{1}$$

$$S_{t,k} = \mathrm{Top3}(g_{t,k}), \tag{2}$$

$$\tilde{g}^{(j)}_{t,k} = \frac{g^{(j)}_{t,k}}{\sum_{\ell\in S_{t,k}} g^{(\ell)}_{t,k}}, \quad j \in S_{t,k}, \tag{3}$$

$$f^k_t = \sum_{j\in S_{t,k}} \tilde{g}^{(j)}_{t,k}\, \mathbf{X}^{(k)}\mathbf{A}_j\mathbf{B}_j. \tag{4}$$

Here σ is a softmax over the experts, and both the top-3 choice and the renormalisation happen separately for every token. We keep the routing in fp32 so that it stays numerically stable when the rest of the model runs in mixed precision.

Shared head adapter: The shared path does not pick experts. It blends all of them with one global gate $\mathbf{z}_s \in \mathbb{R}^N$, which is the same for every token and every slice:

$$f^k_s = \sum_{j=1}^{N} \sigma(\mathbf{z}_s)_j\, \mathbf{X}^{(k)}\mathbf{A}^s_j\mathbf{B}^s_j. \tag{5}$$

C. Shared-Enhanced Task Fusion

After the slices are joined back to width d, the update from the active task is combined with the shared update inside each wrapped layer:

$$\mathbf{Y} = \mathbf{W}_0\mathbf{X} + \beta\left(\mathrm{concat}_k f^k_t + \mathrm{concat}_k f^k_s\right), \tag{6}$$

where $\beta = \alpha/r$ sets the size of the low-rank updates. Both tasks train the shared path, so whatever it picks up from the forgery-type labels also shows up during the authenticity pass. Since this happens in every block, the final CLS token, $\mathbf{h}^{(t)}_{\mathrm{cls}}$, ends up depending on the shared and task-specific updates throughout the network. We start all $\mathbf{B}$ matrices at zero, which means that at the beginning of training the model is exactly the frozen DINOv2.

D. Heads and Losses

The authenticity head outputs $p = \mathrm{sigmoid}(\mathbf{w}_a^\top \mathbf{h}^{(0)}_{\mathrm{cls}} + b_a)$, the probability that the face is fake, and we train it with binary cross-entropy:

$$\mathcal{L}_{\mathrm{bce}} = \mathrm{BCE}(y,\, p). \tag{7}$$

The forgery-type head chooses among $C = 5$ classes (real, Deepfakes, Face2Face, FaceSwap and NeuralTextures) and is trained with cross-entropy:

$$\mathcal{L}_{\mathrm{ftc}} = -\sum_{c=1}^{C} y_c \log \mathrm{softmax}\Big(\mathbf{W}_f\mathbf{h}^{(1)}_{\mathrm{cls}} + \mathbf{b}_f\Big)_c. \tag{8}$$

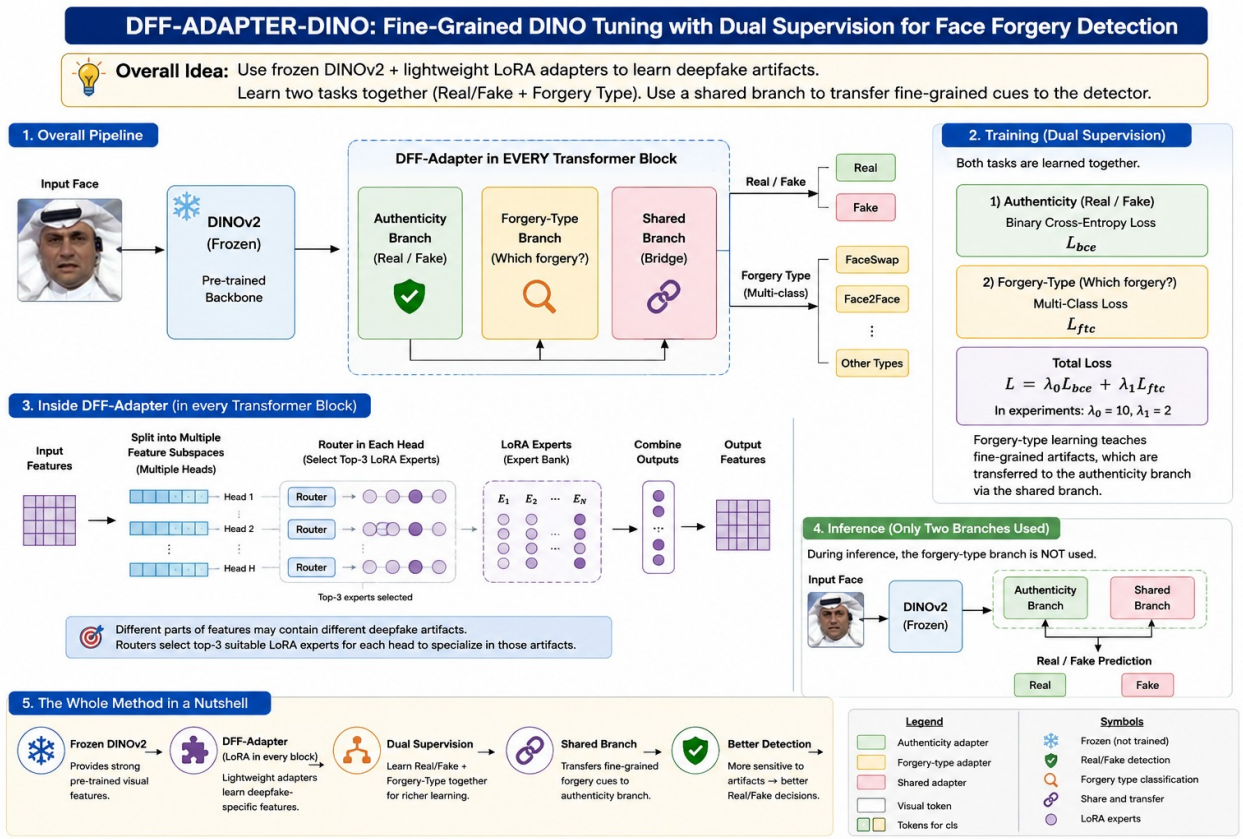


Fig. 1. Overview of the DFF-Adapter pipeline (figure from our project proposal). A frozen DINOv2 backbone carries authenticity, forgery-type and shared adapter branches in every Transformer block. Both tasks are trained together; at inference only the authenticity and shared branches are used.

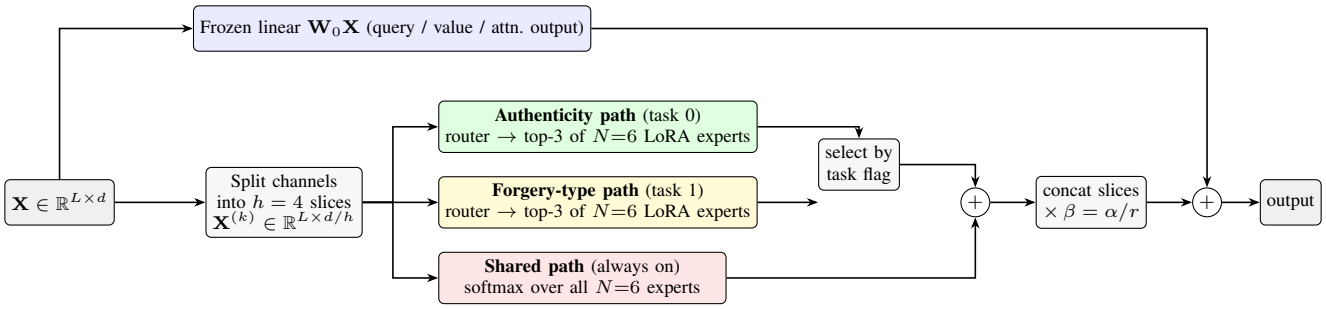


Fig. 2. One DFF-Adapter module in our implementation. The module sits around a frozen square linear layer in the attention of each Transformer block. Inside each path, the input is cut into h channel slices, and every slice goes through the same bank of N LoRA experts ($d_h \to r_h \to d_h$). The two task paths pick the top-3 experts per slice with a router that looks at each token, whereas the shared path blends all experts using one learned global gate. We add the active task update to the shared update, scale the sum by β, and add it to the frozen output as a residual.

E. Dual-Branch Training

Each mini-batch passes through the frozen backbone twice. With the task flag at 0 we get the authenticity logits, and with the flag at 1 we get the forgery-type logits. The two losses are combined as

$$\mathcal{L} = \lambda_0 \mathcal{L}_{\text{bce}} + \lambda_1 \mathcal{L}_{\text{ftc}}, \qquad \lambda_0 = 10,\ \lambda_1 = 2. \tag{9}$$

To keep memory use down, we back-propagate $\lambda_0 \mathcal{L}_{\text{bce}}$ straight after the first pass and $\lambda_1 \mathcal{L}_{\text{ftc}}$ after the second, and only then take an optimiser step. Gradients simply add up, so the result is the same as back-propagating $\mathcal{L}$ in one go, but we only ever hold the activations of one pass in memory. The optimiser touches nothing except the adapters and the two heads.

F. Differences from the Paper

Some details are not fully spelled out in the paper, so we had to make our own choices; Table I lists them. Three matter most. First, our task routers look at each token, while the paper describes a fixed table of routing logits, $\mathbf{Z}_{\text{task}}$. Second, we combine the shared and task updates by adding them inside each wrapped layer (6), whereas the paper writes the fusion as a concatenation at the CLS token. Third, our forgery-type head also has a class for real faces.

TABLE I
HOW OUR IMPLEMENTATION COMPARES WITH THE PAPER

Aspect	Paper	Ours
Task routing	static logit table, top-3	per-token router + bias, top-3
Shared routing	global softmax over N	same
Fusion	concat. at CLS token	residual sum in each layer
Wrapped layers	query, value, dense	query, value, attn. output
Type classes	forgery methods	4 methods + real (5)
LR schedule	not stated	cosine decay to 0
Precision	not stated	fp16 AMP, fp32 routing

TABLE II
FF++ (C23) DATA USED, AS COUNTED BY OUR LOADER

	Train	Val	Test
Video pairs	360	70	70
Real videos found	719/720	140/140	140/140
Fake videos per method	719–720/720	140/140	140/140
Frames per video	8	16	32
Total frames	28,768	11,186	22,388

IV. EXPERIMENTAL SETUP

A. Dataset

FF++ [14] has 1,000 real YouTube videos and 4,000 fakes, made with Deepfakes, Face2Face, FaceSwap and NeuralTextures. We requested access through the official form and, once approved, downloaded the c23 videos (H.264 with a constant rate factor of 23) using the official script (`-d all -c c23 -t videos`).

B. Face Preprocessing

For this step we relied on the DeepfakeBench [18] preprocessing script. On each sampled frame it finds faces with dlib's frontal detector [19] and keeps the largest one. It then takes five keypoints (the eyes, the nose and the corners of the mouth) from the 81-point landmark model, aligns the face with a similarity transform, and writes a 256×256 crop with a 1.3 scale margin into a `frames/` folder. Landmarks and masks are saved next to it. We then uploaded the processed data as a private Kaggle dataset for our training notebook.

C. Splits and Sampling

Our loader works on the face crops directly and follows the official FF++ `train/val/test.json` video pairs (360/70/70 pairs). A pair (a, b) stands for the real videos a and b and, for each method, the fakes `a_b` and `b_a`. The split is therefore made by video and by identity, and no frame can leak from one split into another. We sample 8 evenly spaced frames from each training video, 16 from each validation video and 32 from each test video (Table II). In the training set, one video per class had no usable face crops; FaceSwap is the only class with nothing missing.

A few parts of the pipeline did not feed into the results reported here. The FaceForensics frame extractor saves full frames to `images/`, which our loader never reads, and the landmarks and masks saved by DeepfakeBench are not used by the model either.

D. Evaluation Protocols

We evaluate the ViT-L model in three settings, all trained only on FF++ (c23) and all following the DeepfakeBench protocol used in the paper.

- **Intra-dataset:** the held-out FF++ test split (70 pairs, 700 videos), with 32 frames per video.
- **Cross-dataset:** the test sets of Celeb-DF v1 and v2 [15], the DFDC preview (DFDCP) and the full DFDC [16]. These were preprocessed with the same DeepfakeBench scripts and loaded through the JSON files written by its `rearrange.py` (our `--data json` mode).
- **Cross-manipulation:** twelve methods from DF40 [17], the same twelve the paper reports, split into face swapping (FaceDancer, InSwapper, FSGAN, UniFace), face reenactment (FOMM, HyperReenact, Wav2Lip, MCNet) and entire face synthesis (StyleGAN3, StyleGAN-XL, VQGAN, DiT-XL/2).

E. Implementation Details

Like the paper, we use the `facebook/dinov2-with-registers-large` checkpoint as our backbone, and we also tried the smaller `-base` version. A DFF-Adapter is placed on the query, value and attention-output projections of every block, which gives 72 wrapped layers for ViT-L (24 blocks) and 36 for ViT-B (12 blocks). The settings are rank $r = 16$, $\alpha = 32$ ($\beta = 2$), $N = 6$ experts and $h = 4$ head adapters ($r_h = 4$). Images are resized to 224×224 and normalised with ImageNet statistics, and during training we add random horizontal flips and colour jitter (0.1). We train with Adam [20] (learning rate 2×10^{-4}, weight decay 10^{-5}) with a cosine learning-rate schedule that decays to zero over the iterations, batch size 24, gradient clipping at norm 5, fp16 mixed precision and seed 0. An epoch has 1,198 iterations, and we keep the checkpoint with the best video-level AUC on the validation set.

F. Hardware and Software

All training was done in Kaggle GPU notebooks, and the final FF++ test-split evaluation was run on a department workstation at IIT Bhilai. The code uses Python 3, PyTorch, Hugging Face Transformers and scikit-learn; preprocessing additionally uses OpenCV and dlib.

G. Evaluation Metric

As in the paper and in DeepfakeBench, our main metric is video-level AUC: we average the frame probabilities of each video and then compute the ROC AUC over videos. We also record the frame-level AUC.

H. Parameter Count

Table III splits up the trainable parameters, counted directly from our saved checkpoints. For ViT-L we get 3.549M in total, which agrees with the 3.5M given in the paper.

TABLE III
TRAINABLE PARAMETERS, COUNTED FROM OUR SAVED CHECKPOINTS

Component	ViT-B	ViT-L
Hidden width d / blocks	768 / 12	1024 / 24
Wrapped linear layers	36	72
Authenticity paths	498,528	1,328,832
Forgery-type paths	498,528	1,328,832
Shared paths	331,992	885,168
Authenticity head	769	1,025
Forgery-type head	3,845	5,125
Total trainable	**1,333,662**	**3,548,982**

TABLE IV
FF++ (C23) INTRA-DATASET AUC (%)

Source	Backbone	Epochs	Split	Video	Frame
Paper [1], Tab. 6	ViT-B	50	test	99.61	–
Paper [1], Tab. 1	ViT-L	50	test	99.56	–
Ours	ViT-B	12	val	99.55	98.66
Ours	ViT-L	50	val	99.69	–
Ours	ViT-L	50	test	**99.62**	**98.97**

Validation rows show the best epoch of each run. The frame-level AUC of the best ViT-L validation epoch was not logged.

V. RESULTS AND ANALYSIS

A. Intra-Dataset Results

Table IV puts our runs next to the paper's numbers. On the held-out FF++ test split, our ViT-L model reaches a video-level AUC of 99.62% and a frame-level AUC of 98.97%. The paper reports 99.56% for the same backbone, so we match it, and in fact sit 0.06 points above it. During training, the best validation AUC was 99.69% for ViT-L after 50 epochs and 99.55% for ViT-B after just 12 epochs. The test score is only slightly below the validation score, which tells us that picking the checkpoint on the validation split did not inflate our numbers by much. Our first target, an intra-dataset AUC of about 99%, is therefore clearly met.

B. Training Dynamics

The 12-epoch ViT-B run is shown in Table V and Fig. 3. After just one epoch the validation video AUC is already 99.00%. It reaches its peak of 99.55% at epoch 8 and then moves by less than 0.1 points, even though both weighted training losses keep going down. In other words, the model learns FF++ very quickly, and the later epochs mostly fit the training data more closely. An epoch took about 14 minutes, or 0.64 s per iteration with two forward passes each.

C. Cross-Dataset Generalisation

Table VI shows how the FF++-trained ViT-L model does on four datasets it never saw. It averages 88.00% video-level AUC, against 93.23% in the paper, so it falls 5.24 points short of our second target. The gap is very uneven. On DFDCP we are within a point of the paper (90.59% against 91.57%), on CDF-v2 and DFDC we are about 5 and 7 points behind, and on CDF-v1 we are almost 8 points behind (88.20% against 96.14%). For context, the table also lists two detectors from the paper's own comparison. Our model is clearly ahead of F3Net [6] and level with SBI [7], a strong detector built specifically for generalisation, and on DFDC it beats SBI by 8.62 points.

TABLE V
PER-EPOCH LOG OF THE VIT-B RUN

Epoch	$\lambda_0\mathcal{L}_{\text{bce}}$	$\lambda_1\mathcal{L}_{\text{ftc}}$	Video AUC	Frame AUC	Min
1	2.862	1.165	99.00	97.42	14.7
2	1.254	0.325	99.36	98.23	14.1
3	0.947	0.226	99.35	98.36	14.1
4	0.673	0.171	99.46	98.54	14.0
5	0.507	0.133	99.40	98.37	14.0
6	0.330	0.092	99.34	98.38	14.0
7	0.288	0.074	99.45	98.50	14.0
8	0.191	0.051	**99.55**	**98.66**	13.9
9	0.142	0.043	99.47	98.56	13.9
10	0.095	0.033	99.48	98.58	13.9
11	0.074	0.030	99.49	98.57	13.9
12	0.063	0.026	99.49	98.55	13.9

Losses are running averages of the weighted terms over the first 1,100 iterations of each epoch. AUCs (%) are measured on the FF++ validation split.

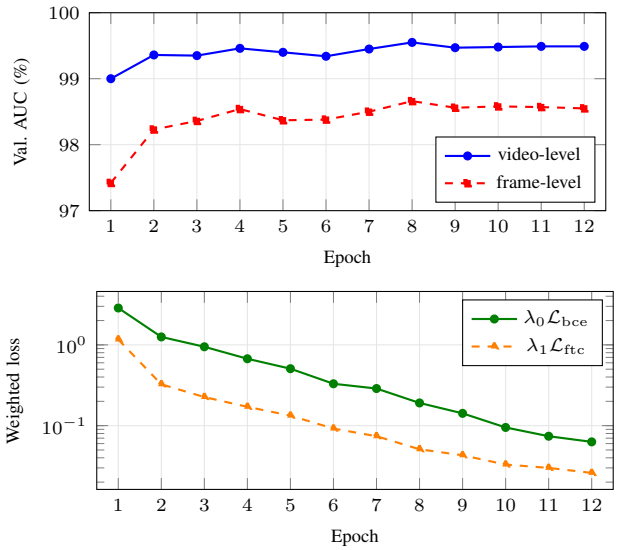


Fig. 3. ViT-B run. Top: validation AUC after each epoch. Bottom: weighted training losses on a log scale.

TABLE VI
CROSS-DATASET VIDEO-LEVEL AUC (%), TRAINED ON FF++ ONLY

Method	CDF-v2	DFDC	CDF-v1	DFDCP	Avg.
F3Net [6]	78.88	71.77	81.11	73.50	76.32
SBI [7]	93.82	74.47	93.44	90.95	88.17
DFF-Adapter (paper)	95.26	89.96	96.14	91.57	93.23
Ours (video level)	90.11	83.09	88.20	90.59	88.00
Ours (frame level)	83.63	81.48	80.97	86.35	83.11
Ours − paper (video)	−5.15	−6.87	−7.94	−0.98	−5.24

F3Net, SBI and DFF-Adapter values are taken from Table 1 of [1].

TABLE VII
CROSS-MANIPULATION VIDEO-LEVEL AUC (%) ON DF40

Group	Method	Ours (video)	Ours (frame)	Paper (video)	Diff.
FS	FaceDancer	92.30	84.83	93.15	−0.85
	InSwapper	98.17	95.19	93.98	+4.19
	FSGAN	97.30	95.50	98.84	−1.54
	UniFace	97.94	97.20	94.51	+3.43
FR	FOMM	95.74	90.73	95.36	+0.38
	HyperReenact	75.78	73.12	90.28	−14.50
	Wav2Lip	95.14	90.54	91.65	+3.49
	MCNet	83.72	79.23	86.29	−2.57
EFS	StyleGAN3	98.08	93.26	99.92	−1.84
	StyleGAN-XL	99.81	98.26	100.00	−0.19
	VQGAN	99.45	97.76	100.00	−0.55
	DiT-XL/2	96.50	82.20	98.44	−1.94
FS average		96.43	93.18	95.12	+1.31
FR average		87.59	83.40	90.90	−3.31
EFS average		98.46	92.87	99.59	−1.13
Overall average		**94.16**	**89.82**	**95.20**	**−1.04**

FS: face swapping; FR: face reenactment; EFS: entire face synthesis. Paper values are from Table 2 of [1]. Diff. is ours minus paper, video level.

D. Cross-Manipulation Results on DF40

Table VII compares our model with the paper on twelve DF40 methods. Here the picture is much closer. Our average video-level AUC is 94.16% against the paper's 95.20%, and we are ahead on four of the twelve methods: InSwapper (+4.19), Wav2Lip (+3.49), UniFace (+3.43) and FOMM (+0.38). Face swapping is our strongest group, where we beat the paper on average (96.43% against 95.12%). Entire face synthesis is nearly solved by both models (98.46% against 99.59%). Face reenactment is the weak spot: the group averages 87.59% against the paper's 90.90%, and almost all of that comes from HyperReenact, where we score 75.78%, 14.50 points below the paper. MCNet is our second-lowest method at 83.72%.

E. Video Level Versus Frame Level

Averaging over frames matters much more on unfamiliar data. On the FF++ test split, video-level AUC is only 0.65 points above frame-level AUC. Across the four cross-dataset benchmarks the gap grows to 4.89 points on average, and on DF40 to 4.34 points, with DiT-XL/2 showing the largest gap (96.50% against 82.20%). In other words, when the model meets a new kind of fake, many single frames are judged wrongly, but the errors are scattered enough that the average over a video is still right much more often.

F. What the Adapters Learned

To see what the adapters learned, we looked directly at the weights stored in the two best checkpoints.

The shared gate stays almost uniform: Across all wrapped layers, the shared softmax gate $\sigma(\mathbf{z}_s)$ gives each expert close to $1/6$ of the weight (between 0.157 and 0.181 for ViT-B, and between 0.151 and 0.185 for ViT-L, with normalised entropy ≥ 0.998). In practice, then, the shared path acts as an even sum of its six experts. It learns through the expert matrices themselves, not by preferring some experts over others.

Routing is driven by the input: In the task paths, the fixed router biases $\mathbf{z}_{t,k}$ remain small ($|\mathbf{z}| \leq 0.1$). Which experts get chosen is therefore decided almost entirely by the token-dependent term $\mathbf{X}^{(k)}\mathbf{W}^r_{t,k}$. We see this as support for using an input-dependent router, because a fixed table alone would have led to routing that barely changes.

Deeper blocks adapt more: The average Frobenius norm of the up-projections $\mathbf{B}_j$ gets larger as we go deeper into the network. In ViT-L, the authenticity path goes from 0.98 in the first block to around 2.1–2.2 in the middle and final blocks, and the other paths and ViT-B follow the same pattern. Still, no block is left untouched, which fits the paper's case for adapting every block and not just the last one.

Training stability: Because of fp16 overflow, the gradient scaler skipped 10 of the 14,376 updates for ViT-B and 25 of the 59,900 for ViT-L. That is less than 0.1% of all updates, and neither run diverged.

TABLE VIII
BEST VALIDATION VIDEO AUC (%) IF VIT-B TRAINING HAD STOPPED AFTER k EPOCHS

k (epochs)	1	2	4	8	12
Best AUC so far	99.00	99.36	99.46	99.55	99.55

VI. ABLATION STUDY

With the compute we had, we trained two full models. This section first reports the comparisons that these two runs do support, then sets them against the component ablations published by the authors, uses those ablations to read our own cross-dataset results, and finally lists the ablations we plan to run ourselves.

A. Backbone Size and Schedule

Going from ViT-B for 12 epochs to ViT-L for 50 epochs raises the trainable parameters from 1.33M to 3.55M and the best validation AUC from 99.55% to 99.69% (Table IV). The gain is small, and the two runs differ in both backbone and training length, so we cannot say which change caused it. It does show, though, that the smaller and cheaper model already gets most of the way on FF++.

B. Training Length

Our ViT-B log lets us ask a simple question: what would the best model have been if training had stopped earlier? Table VIII answers it. Stopping after a single epoch would already give 99.00%, four epochs give 99.46%, and all later epochs together add only 0.09 points. On FF++, longer training buys very little.

TABLE IX
COMPONENT ABLATION REPORTED BY ZHANG ET AL. [1]: CROSS-DATASET VIDEO AUC (%)

Configuration	CDF-v2	DFDC	CDF-v1	DFDCP
DINOv2 only	61.93	52.83	68.46	55.54
+ multi-head router	89.56	86.24	85.53	87.11
+ shared-enhanced fusion	95.26	89.96	96.14	91.57

Values copied from Table 3 of [1]; we have not re-run these experiments.

C. Component Ablations Reported in the Paper

The authors removed parts of the model one at a time and measured the effect on unseen datasets [1]. Their results, summarised in Table IX, are worth reading next to ours. A plain DINOv2 averages only about 60% AUC across the four test sets. Adding the multi-head router lifts this to about 87%, and adding the shared-enhanced fusion on top brings it to about 93%. Their loss-weight study is just as telling: with the forgery-type loss switched off ($\lambda_1 = 0$), FF++ AUC is actually slightly higher (99.65%), but CDF-v2 drops from 95.26% to 90.93% and DFDC from 89.96% to 84.44%. In other words, the second task mainly pays off on data the model has not seen. Their head and expert studies also favour the settings we used ($h = 4$, $N = 6$).

D. Our Results Next to the Paper's Loss Ablation

Putting our cross-dataset numbers next to the paper's loss ablation gives a useful hint. On CDF-v2 and DFDC, the two datasets the paper reports for this ablation, our full model scores 90.11% and 83.09%. The paper's variant trained *without* the forgery-type loss scores 90.93% and 84.44%, while its full model scores 95.26% and 89.96%. Our numbers are therefore much closer to the paper's single-task variant than to its full model, even though we do train with the type loss. On FF++, by contrast, the paper's two variants are almost identical. One reading is that, in our version, the second task is not passing its benefit on to unseen data as strongly as it does for the authors. The places where our design differs from theirs (Table I), namely the per-token router, the extra real class in the type head and where the shared update is fused, are the first suspects. This is a hypothesis, not a result: only the ablations below can confirm it.

E. Planned Ablations

We have not yet re-run any of the component ablations ourselves. Table X lists the ones we plan to run first, each trained on FF++ and tested on the cross-dataset benchmarks, since that is where both the paper and our own results say the differences show up. The first two rows directly test the hypothesis above.

TABLE X
ABLATIONS PLANNED FOR OUR IMPLEMENTATION

Change	Question it answers
$\lambda_1 = 0$ (no type loss)	Does the second task help us at all?
Static routing (bias $\mathbf{z}$ only)	Does the paper's fixed table generalise better than our per-token router?
Remove the shared path	How much does the bridge between tasks add?
Type head without the real class	Does matching the paper's label set change the transfer?
Top-$k \in \{1, 3, 6\}$	How sparse should routing be?
$h \in \{1, 4, 8\}$ heads	Does our code reproduce the paper's choice of $h = 4$?

VII. DISCUSSION

What reproduced and what did not: The in-domain result reproduces fully: 99.62% on the FF++ test split against the paper's 99.56%. The cross-manipulation result comes close, at about one point below the paper on average, and we even do better on four DF40 methods. The cross-dataset result does not fully reproduce: we are 5.24 points short on average, with the biggest gaps on CDF-v1 and DFDC. Because FF++ is close to saturated, with every DFF-Adapter variant in the paper's ablations landing between 99.2% and 99.7%, a matching in-domain score shows that our implementation works but cannot tell good generalisation from great generalisation. The cross-dataset tests are where the real differences appear.

Where the cross-dataset gap may come from: Three explanations seem plausible, and they are not mutually exclusive. First, our design choices (Table I) may weaken the transfer from the forgery-type task, as Section VI suggests. Second, we choose the checkpoint by FF++ validation AUC, which rewards in-domain fit. Our training log shows that this score stops improving early, so later epochs may make the model more specific to FF++ without any sign of it on the validation split. Third, small differences in preprocessing, frame sampling and the exact test lists can move cross-dataset AUC by a few points, and with one run and one seed we cannot separate these effects.

Why reenactment is harder: Our weakest group on DF40 is face reenactment, and HyperReenact in particular. Reenactment keeps the identity of the person and only changes expression or pose, so it leaves fewer and subtler traces than a face swap. FF++ only contains two methods of this kind (Face2Face and NeuralTextures), which gives the model little to learn from. It is encouraging that we still match or beat the paper on FOMM and Wav2Lip.

The shared branch behaves like a plain LoRA ensemble: The near-uniform shared gate suggests that the shared path does not need its gating at all: it could be replaced by a single wider LoRA with little change. Its value, if the paper's ablation carries over to our version, lies in being trained by both tasks rather than in how it mixes its experts.

Per-token routing: flexible, but possibly too flexible: Because the learned router biases stay close to zero, a purely static routing table would have given almost the same experts for every input. Our per-token router lets expert choice depend on the content of each patch, which may help it fit FF++ very closely. Given our cross-dataset gap, it is also possible that this extra freedom makes the model more dataset-specific. The static-routing ablation will tell us which effect dominates.

Cost: Training is about twice as expensive as a single-task model because each batch goes through the backbone twice, but inference costs a single pass. Only the 3.55M adapter and head parameters need to be stored per trained model, against roughly 300M frozen backbone weights that can be shared.

Practical lessons: Most of our time went into data rather than modelling: getting dataset access, running the face preprocessing for several datasets, and making our copies match the official splits. Checking frame counts before training (our loader's `check` mode) caught missing videos early, and saving only the trainable tensors kept checkpoints small enough (5–14 MB) to move around easily.

VIII. Limitations

- **No ablations of our own yet.** The component-level evidence in Section VI comes from the paper, not from our code, and our explanation of the gap is still a hypothesis. We also did not run identity-constrained training or the t-SNE and Grad-CAM visualisations.
- **Partial DF40 coverage.** We test the twelve DF40 methods that the paper reports in its main table, not all forty, and not the extra synthesis methods (RDDM, PixArt-α, SiT-XL/2) in its appendix.
- **Single metric.** We report AUC only. EER, which the paper also reports, is not yet computed.

IX. Conclusion and Future Work

Working only from the paper, we rebuilt the DeepFake Fine-Grained Adapter for DINOv2 and set up a pipeline that can be rerun end to end, from downloading FF++ to testing on six benchmarks. With the same ViT-L backbone and 50-epoch schedule as the paper, our model uses exactly the reported 3.5M trainable parameters and reaches 99.62% video-level AUC on the FF++ test split, matching the paper's 99.56%. It also holds up well against new manipulation methods, averaging 94.16% on twelve DF40 methods against the paper's 95.20%. On completely unseen datasets it averages 88.00%, which is level with strong published detectors such as SBI but 5.24 points below the paper. A frozen DINOv2 tuned with routed low-rank experts and two kinds of supervision is clearly a strong and cheap detector; whether it generalises as well as the paper claims depends on details our reproduction has not yet pinned down.

For future work, we will (1) run the ablations in Table X, starting with $\lambda_1 = 0$ and static routing, to find out where the cross-dataset gap comes from; (2) repeat the main runs and add EER; and (3) extend the DF40 evaluation to the remaining methods, with a closer look at face reenactment.

Acknowledgments

We are grateful to the FaceForensics++ authors for giving us access to their dataset, to the DeepfakeBench team for their preprocessing tools, and to Kaggle for the GPU time.

References

[1] T. Zhang, P. Yu, Z. Xia, L. Dai, X. Zhou, and H. Gao, "Fine-grained DINO tuning with dual supervision for face forgery detection," *arXiv preprint arXiv:2511.12107*, 2025.

[2] R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales, and J. Ortega-Garcia, "Deepfakes and beyond: A survey of face manipulation and fake detection," *Information Fusion*, vol. 64, pp. 131–148, 2020.

[3] M. Masood, M. Nawaz, K. M. Malik, A. Javed, A. Irtaza, and H. Malik, "Deepfakes generation and detection: State-of-the-art, open challenges, countermeasures, and way forward," *Applied Intelligence*, vol. 53, no. 4, pp. 3974–4026, 2023.

[4] M. Oquab, T. Darcet, T. Moutakanni, H. Vo *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[5] L. Li, J. Bao, T. Zhang, H. Yang, D. Chen, F. Wen, and B. Guo, "Face X-ray for more general face forgery detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 5001–5010.

[6] Y. Qian, G. Yin, L. Sheng, Z. Chen, and J. Shao, "Thinking in frequency: Face forgery detection by mining frequency-aware clues," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020, pp. 86–103.

[7] K. Shiohara and T. Yamasaki, "Detecting deepfakes with self-blended images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 18 720–18 729.

[8] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[10] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski, "Vision transformers need registers," in *International Conference on Learning Representations (ICLR)*, 2024.

[11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[12] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations (ICLR)*, 2017.

[13] C. Kong, A. Luo, P. Bao, Y. Yu, H. Li, Z. Zheng, S. Wang, and A. C. Kot, "MoE-FFD: Mixture of experts for generalized and parameter-efficient face forgery detection," *arXiv preprint arXiv:2404.08452*, 2024.

[14] A. Rossler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1–11.

[15] Y. Li, X. Yang, P. Sun, H. Qi, and S. Lyu, "Celeb-DF: A large-scale challenging dataset for deepfake forensics," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 3207–3216.

[16] B. Dolhansky, J. Bitton, B. Pflaum, J. Lu, R. Howes, M. Wang, and C. C. Ferrer, "The DeepFake detection challenge (DFDC) dataset," *arXiv preprint arXiv:2006.07397*, 2020.

[17] Z. Yan, T. Yao, S. Chen, Y. Zhao, X. Fu *et al.*, "DF40: Toward next-generation deepfake detection," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 29 387–29 434.

[18] Z. Yan, Y. Zhang, X. Yuan, S. Lyu, and B. Wu, "DeepfakeBench: A comprehensive benchmark of deepfake detection," *arXiv preprint arXiv:2307.01426*, 2023.

[19] D. E. King, "Dlib-ml: A machine learning toolkit," *Journal of Machine Learning Research*, vol. 10, pp. 1755–1758, 2009.

[20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

APPENDIX A
AI PROMPTS GIVEN TO LLM

A. Understanding and Planning

a) I'm reading the paper "Fine-Grained DINO Tuning with Dual Supervision for Face Forgery Detection" (arXiv:2511.12107). Can you explain in simple terms what the DFF-Adapter is and how it differs from normal LoRA? I didn't understand the routed experts part.

b) I want to implement this paper using DINOv2 with registers (large) as a frozen backbone. Where exactly should the adapters go in each ViT block? The paper says q, v and attention output projections, so how do I do that with the Hugging Face DINOv2 model?

B. Model Code

a) Give me code to load `facebook/dinov2-with-registers-large` from Hugging Face, freeze all its parameters, and print how many parameters are trainable versus total. I want to confirm that only the adapters will be trained.

b) Write a PyTorch module for a low-rank adapter with rank 16 and alpha 32 that has 6 experts and a router that picks between them. It should wrap an existing `nn.Linear` layer so the original weights stay frozen.

c) Now write a function that goes through all the Transformer blocks and replaces the query, value and attention output linear layers with my adapter version. I don't know the exact layer names in the Hugging Face DINOv2 implementation, so please check those too.

d) Add two heads on top of the CLS token, one with 2 outputs for real/fake and one with 4 outputs for Deepfakes, Face2Face, FaceSwap and NeuralTextures. Put everything in a single model class in `model.py` that returns both logits.

e) The paper uses two losses, real versus fake and forgery method. Real images don't have a method label, so I was thinking of giving them -1. Will that work with `CrossEntropyLoss` using `ignore_index`, and how do I weight the two losses 10:2?

C. Data Preparation

a) I'm using FaceForensics++ c23 from DeepfakeBench. I'm worried about data leakage, so how should I split the data so frames from the same video don't end up in different splits?

b) Write `scripts/build_ffpp_manifest.py`. It should walk through the FF++ c23 folders (original and the four manipulated types), sample up to 32 frames per video, and write train, validation and test JSONL files with `path`, `label`, `type_label` and `video_id`. If official split JSON files are given, it should use them; otherwise, use an 80/10/10 split by video.

c) Write a PyTorch Dataset class that reads this JSONL manifest, loads the face image, resizes it to 224×224, normalizes it with ImageNet mean and standard deviation, and returns the image, label, type label and video ID.

D. Training

a) Make a YAML configuration for FF++ c23 training with all hyperparameters, including batch size, gradient accumulation steps, rank, alpha, number of experts, number of heads, loss weights, epochs, manifest paths and output folder.

b) Write `train.py` that reads this YAML configuration, builds the data loaders, and trains with Adam (learning rate 2×10^{-4} and weight decay 10^{-5}) for 50 epochs. Use mixed precision and the combined loss defined earlier.

c) I only have a Kaggle T4 GPU with 16 GB of memory and get CUDA out of memory with batch size 24. How can I use gradient accumulation to keep the same effective batch size? Please update the training loop.

d) Kaggle sessions keep disconnecting. Update the training script to save `last.pt` after every epoch and `best.pt` based on validation video-level AUC, write metrics to `history.jsonl`, and support resuming training with the optimizer state and epoch number.

e) My training crashes after the first epoch with a shape mismatch in the router. Here is my adapter code and the error. Can you find the bug and provide the corrected code?

E. Evaluation

a) The paper reports video-level AUC, but my model predicts per frame. How do I average the frame scores per video and calculate AUC with scikit-learn?

b) Write `evaluate.py` that loads a checkpoint, runs evaluation on a manifest, and prints frame-level AUC, video-level AUC and accuracy.

c) Add an optional `--manifest` argument to `evaluate.py` so I can test the same checkpoint on other datasets, such as Celeb-DF, without retraining.

F. Code Cleanup and Documentation

a) Refactor my code to make it cleaner, add comments and docstrings, and create a `requirements.txt` with appropriate versions of PyTorch, Transformers and scikit-learn.
b) Help me write a README explaining how to run this project on Kaggle step by step, and mention that it is my own implementation of the paper because the authors didn't release official code.